

Chapter 4. Financial Entity Systems

In the last chapter, you learned about financial identifiers and identification systems and their critical role in financial markets. Importantly, before a financial entity can be identified, it must first be extracted and ready for identification. However, in finance, it's quite common for data to exist in an unstructured format, where entities are not immediately identifiable. In fact, analysts [estimate](#) that the vast majority of data in the world exists in unstructured formats, such as text, video, audio, and images. Moreover, it is quite frequent that different identifiers are used to reference the same financial entity across both structured and unstructured data. These factors collectively pose significant challenges when trying to extract value and insights from the data.

To this end, many financial institutions develop systems to extract, recognize, identify, and match financial entities within financial datasets. These systems, which I will call *financial entity systems* (FESs), constitute the main topic of this chapter. As a financial data engineer, understanding FESs and the challenges they entail is essential in navigating today's complex financial data landscape.

In the first part of this chapter, I will clarify the notion of financial entities and provide an overview of their various types. Next, I will illustrate the problem of financial entity extraction and recognition using a popular FES called *named entity recognition*. After that, I'll cover the issue of financial data matching and record linkage using another FES known as *entity resolution*.

Financial Entity Defined

Generally speaking, the term *entity* refers to any real-world object that can be recognized and identified. By narrowing the scope to financial markets, we can use the term *financial entity* to denote any real-world entity operating within financial markets. In this book, I define financial entity and financial entity systems as follows:

A financial entity is a real-world object that may be recognized, identified, referenced, or mentioned as an essential part of financial market operations, activities, reports, events, or news. A financial entity may be human or not. It can be tangible (e.g., an ATM machine), intangible (e.g., common stock), fungible (e.g., one-dollar bills), or infungible (e.g., loans). A financial entity system is an organized set of technologies, procedures, and methods for extracting, identifying, linking, storing, and retrieving financial entities and related information from different sources of financial data and content.

As financial markets evolve and expand, so do the diversity and types of financial entities. A frequently used [benchmark classification system](#) categorizes entities into four main groups: individuals (PER), corporations (ORG), places (LOC), and miscellaneous entities (MISC).

Naturally, based on your institution's needs, it might be necessary to categorize entities into a broader or more granular range. For example, let's say that your financial institution decides to collect data on the digital asset market. In this case, you might want to create a new entity type (digital asset) to represent objects such as cryptocurrencies, digital currency, utility tokens, security tokens, stablecoins, bitcoin, and many more. Other examples include the following:

- *Persons*, e.g., bankers, traders, directors, account holders, investors, market makers, regulators, brokers, financial advisors
- *Locations*, e.g., New York, Japan, Africa, Benelux (Belgium, the Netherlands, and Luxembourg)
- *Nationalities*, e.g., Italian, Australian, Chinese

- *Companies*, e.g., Bloomberg L.P., JPMorgan Chase & Co., Aramco, Ferrero
- *Organizations*, e.g., Securities and Exchange Commission, European Central Bank, London Stock Exchange, International Monetary Fund
- *Sectors*, e.g., financial services, food industry, agriculture, construction, microchips
- *Currency*, e.g., dollar (\$), pound (£), euro (€)
- *Commodity*, e.g., gold, copper, silver, wheat, coffee, oil, steel
- *Financial security*, e.g., stocks, bonds, derivatives
- *Corporate events*, e.g., mergers, acquisitions, leveraged buyouts, syndicated loans, alliances, partnerships
- *Financial variables*, e.g., interest rate, inflation, volatility, index value, rating, profits, revenues
- *Investment strategies*, e.g., passive investment, active investment, value investing, growth investing, indexing
- *Corporate and market hierarchies*, e.g., parent company, holding company, subsidiary, branch
- *Products*, e.g., iPhone, Alexa, Siri, Dropbox, Gmail

Now that you know what financial entities are and how to categorize them, let's move on to understand how to identify and extract these entities from financial data. As previously mentioned, the systems designed for this purpose are referred to as *named entity recognition* (NER) systems.

Financial Named Entity Recognition

As a financial data engineer, if you ever get assigned to a project that involves recognizing and identifying financial entities from unstructured or semi-structured text, you will likely design and build an NER system. In this section, I will first define NER and give a few illustrative examples. Then, I will describe how NER works and the steps involved in designing an NER system. Third, I will give an overview of the available methods and techniques for conducting NER. Lastly, I will discuss a few examples of open source and commercial software libraries and tools that you can use to do NER.

Named Entity Recognition Described

NER, also known as entity extraction, entity identification, or entity chunking, is the task of detecting and recognizing named entities in text, such as persons, companies, locations, events, symbols, time, and more. NER is a key problem in finance, given the large volumes of finance-related text generated on a daily basis (e.g., filings, news, reports, logs, communications, messages) combined with the growing demand for advanced strategies for working with unstructured and text data.

The outcome of NER analysis is used in a variety of financial applications, such as enriching financial datasets with entity data, information extraction (e.g., extracting relevant financial information from financial reports and filings), text summarization (e.g., ensuring adherence to legal requirements), fraud detection (identifying suspicious entities and transactions), adverse media screening (i.e., screening an entity against a negative source of information), sentiment analysis (assessing market sentiment from news and social media), risk management (e.g., recognizing potential financial risks and exposures), and extracting actionable insights from financial news, market events, players, competition, trends, and products.

The market for products and services that depend on named entity recognition methods is rapidly expanding. Prominent names in this field include RavenPack, InfoNgen, OptiRisk Systems, and LSEG's Machine Readable News.

[RavenPack News Analytics \(RNA\)](#) is the world-leading news insights and analytics resource. RavenPack collects and analyzes unstructured content from more than 40,000 sources such as Dow Jones Newswires, the *Wall Street Journal*, *Barron's*, MT Newswires, PR Newswire, Alliance News, MarketWatch, The Fly, and providers of regulatory news, press releases, and articles.

RavenPack News Analytics computes 20+ years of point-in-time data and provides event and sentiment data on more than 350,000 entities in over 130 countries, including the following:

- 110,000+ global, public, and private companies across all sectors
- 165,000+ macro entities such as places, currencies, persons, and organizations
- 7,000+ key business and geopolitical and macroeconomic events detected and enriched with sentiment and relevance scores

For each record in RavenPack News Analytics, information is available on:

- The entity (e.g., name, domicile, RavenPack's unique entity identifiers, and other identifiers)
- Event Category
- Event Sentiment Score (how negative or positive an event is [range -1 $\rightarrow$ 1])
- Event Similarity Days (how novel is the event, measured as the number of days passed [range 0 $\rightarrow$ 365] since a similar event occurred)
- Event Relevance Score (how relevant is an event [range 0 $\rightarrow$ 100] based on where it occurs—e.g., a headline has a high ERS)

To identify and extract these relevant aspects from news data, RavenPack built a proprietary named entity recognition system. RavenPack maintains a database of predefined entities with more than 50 distinct entity types to provide timely and high-quality data. Moreover, RavenPack expands and extends its database as new and relevant types of entities or events appear in the market.

In building its NER system, RavenPack faced a special requirement for the financial sector: entity names may change over time, and the same name may refer to different entities at different times. This might lead to problems such as survivorship bias, where only the latest assignee or an identifier or surviving entities are considered, skewing the data and the analysis. To solve this issue, RavenPack constructed a point-in-time-aware NER system.

The main idea behind NER is to take an annotated text such as...

Google has invested more than \$1 Billion in Renewable Energy projects in the United States over the past 5 years

... and produce a new block of text that highlights the position and type of entities, as illustrated in [Figure 4-1](#). In this example, six types of entities are recognized: company, currency, amount, sector, time, and location.

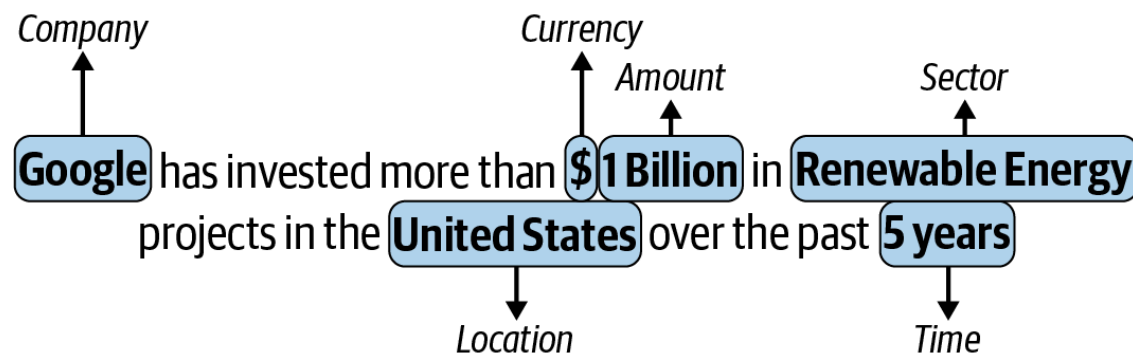


Figure 4-1. An illustration of the outcome of NER

For the sake of illustration, let's walk through a practical example. A well-known financial dataset is [LSEG Loan Pricing Corporation DealScan](#), which offers comprehensive coverage of the syndicated loans market. A syndicated loan (also known as a syndicated facility) is a special type of

loan where a group of lenders (the syndicate) jointly provide a large loan to a company or an organization. Within the syndicate, different agents assume various roles (e.g., participating bank, lead arranger, documentation agent, security agent, etc.). LSEG and similar data providers collect information about syndicated loans from multiple sources, with SEC filings such as 8-Ks as the primary source.

Let's consider a scenario where your team is tasked with creating a dataset on syndicated loans using a collection of SEC filings. Your first step involves extracting data from the text, identifying various elements that characterize a syndicated facility, and then organizing this information into a structured format. Let's take the following example of an SEC filing for a syndicated facility agreement given to an Australian company (the text below is quoted and highlighted from the [SEC filing](#)):

SYNDICATED FACILITY AGREEMENT

dated as of **September 18, 2012**

among

THE MAC SERVICES GROUP PTY LIMITED ,

as Borrower,

THE LENDERS NAMED HEREIN,

J .P. MORGAN AUSTRALIA LIMITED ,

as Australian Agent and Security Trustee ,

JPMORGAN CHASE BANK, N.A. ,

as US Agent ,

JPMORGAN CHASE BANK, N.A.,

as Issuing Bank

and

JPMORGAN CHASE BANK, N.A.,

as Swing Line Lender

J.P. MORGAN SECURITIES LLC ,

as Lead Arranger and Sole Bookrunner

...

The Borrower has requested the Lenders to extend credit, in the form of **Loans or Credits** *(as hereinafter defined), to the Borrower in an*

aggregate principal amount at any time outstanding not in excess of AUD\$300,000,000 .

As you can see, the text includes details regarding the borrower, lenders, and their respective roles, as well as information about the facility type, amount, and currency. Leveraging NER, we can extract this information and construct a structured dataset. For simplicity, let's design a dataset with three tables: one to store facility data, another for borrower details, and a third for lender information. [Figure 4-2](#) shows what the *Entity Relationship Model* of our dataset looks like. In the facility table, the `facility_id` is an arbitrarily assigned unique identifier. In the borrower and lender tables, the `facility_id` is present as a foreign key, meaning that records will exist in these tables only for facilities that exist in the facility table.

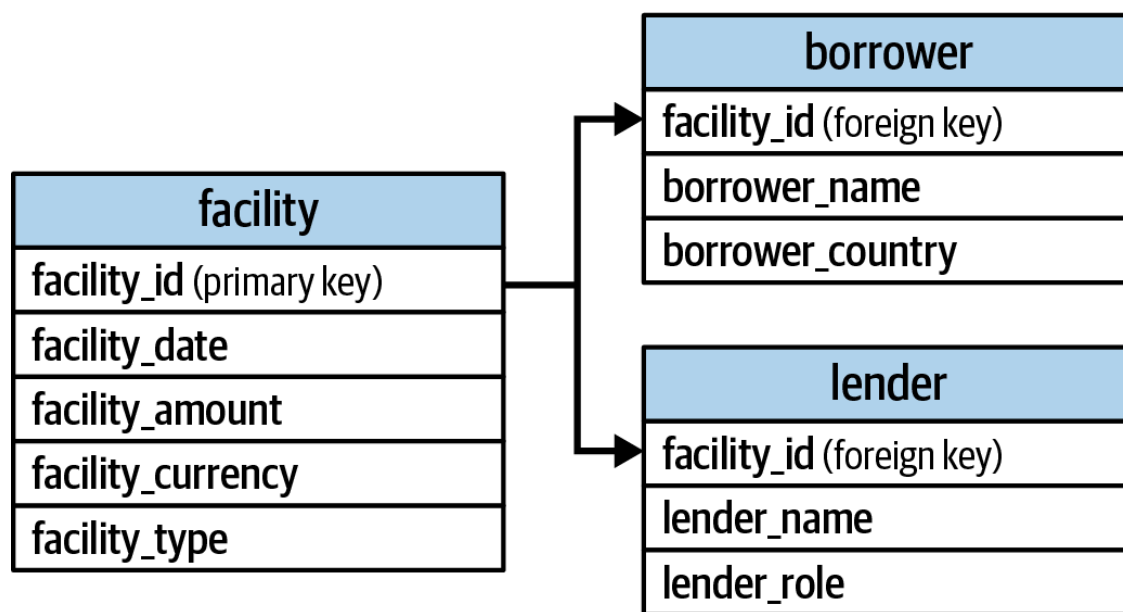


Figure 4-2. Entity Relationship Model (ERM) of the syndicated loan database

The result of a successful NER-based entity extraction would look like the data present in [Tables 4-1](#), [4-2](#), and [4-3](#).

Table 4-1. Facility table

facility_id	facility_date	facility_amount	facility_currency	facility_type
89763	2012-09-18	300,000,000	AUD	Loans or Credits

Table 4-2. Borrower table

facility_id	borrower_name	borrower_country
89763	The Mac Services Group PTY Limited	Australia

Table 4-3. Lender table

facility_id	lender	lender_role
89763	J.P. Morgan Australia Limited	Australian Agent and Security Trustee
89763	JPMorgan Chase Bank, N.A.	US Agent
89763	JPMorgan Chase Bank, N.A.	Issuing Bank
89763	JPMorgan Chase Bank, N.A.	Swing Line Lender
89763	J.P. Morgan Securities LLC	Lead Arranger
89763	J.P. Morgan Securities LLC	Bookrunner

Crucially, although an NER system can identify the occurrence of a specific entity in the text, it typically does not link it to the corresponding real-world object. For example, if you refer back to [Figure 4-1](#), Google was labeled as COMPANY, but at this point, we still don't know which real-world company this is. To accomplish this task, an additional technique, called *named entity disambiguation* (NED) or entity linking, is often used.

Many books treat NED as a separate problem from NER and dedicate a separate section to it. However, for financial applications, linking the identified entities to their real-world matches is essential. For this reason, I consider NED an additional step in the NER process. [Figure 4-3](#) demonstrates how NED works in conjunction with NER to link the recognized entity (COMPANY) to its specific real-world counterpart (Google).

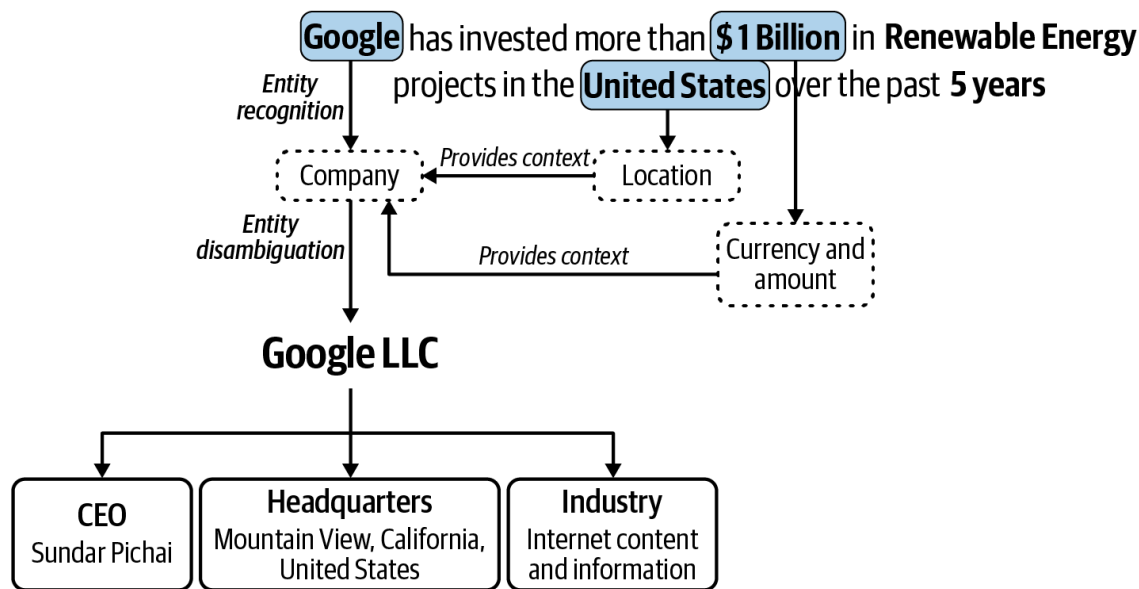


Figure 4-3. Named entity recognition and disambiguation

In NED, entities identified in the text are mapped to their unique real-world counterparts using a *knowledge base*. A knowledge base is a central repository that contains information about a vast array of subjects. These can be general-purpose or specialized and may be public or private. For example, Wikipedia is a well-known public, general-purpose knowledge base, while Investopedia serves a similar role but focuses specifically on finance. Other notable examples include GeoNames, Wikidata, DBpedia, and YAGO. Financial institutions and data vendors may also create proprietary knowledge bases tailored to their specific needs using their own data.

How Does Named Entity Recognition Work?

In this section, we will explore the various steps involved in building an NER system. As illustrated in [Figure 4-4](#), the first step is data preprocessing, which ensures the data is structured, cleaned, harmonized, and ready for analysis. The second step, entity extraction, involves identifying the locations of all candidate entities. In the third step, these candidate entities are categorized into their respective entity types. Subsequently, the quality and completeness of the extracted data and the performance of the model are assessed in the evaluation step. Finally, the recognized entities can optionally be linked to their unique real-world counterparts through the disambiguation process.

Note that NER is an iterative process. Once the model is evaluated, the modeler can determine if improvements in data preprocessing, model selection, or training techniques are necessary to enhance the NER system's performance.

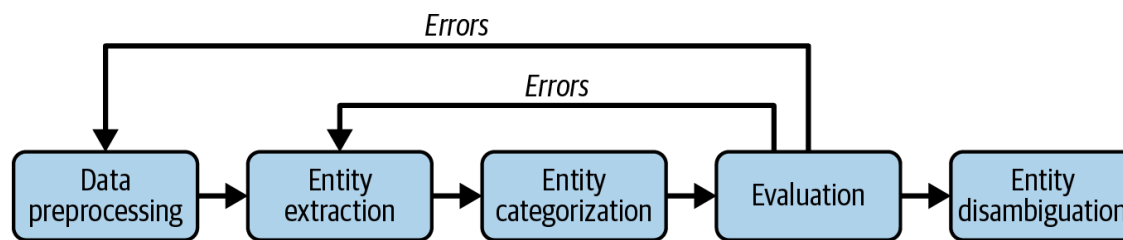


Figure 4-4. Named entity extraction and disambiguation process

Data preprocessing

Methodologically speaking, NER is a subtask of the field of natural language processing (NLP). As with most NLP tasks, NER achieves good results if applied to clean and high-quality data. A variety of NLP-specific data preparation techniques can be used with NER. These include the following:

Tokenization

Tokenization is the process of breaking down the text into smaller units called *tokens*. Word tokenization breaks down the text into single words; for example, “Google invests in Renewable Energy” becomes [“Google”, “invests”, “in”, “Renewable”, “Energy”]. Sentence tokenization breaks down text into smaller individual sentences; for example, “Google invests in Renewable Energy” gets converted into [“Google”, “invests in”, “Renewable Energy”].

Stop word removal

Stop words are common and frequent words that have very little or no value for modeling or performance. For example, the English words “is,” “the,” and “and” are often classified as stop words. In most NLP tasks, including NER, stop words are filtered out.

Canonicalization

In NLP, the form and conjugation of the word are often of no value. For example, the words “invest, investing, invests, invested” convey the same type of action; therefore, they can all be mapped to their base form, i.e., “invest.” The process of mapping words in a text to their root/base forms is known as *canonicalization*.

Two types of canonicalization techniques are often used: *stemming* and *lemmatization*. Stemming is a heuristic technique that involves removing affixes from a word to produce its stem. This method is quick and efficient but can produce imprecise results, as it often leads to over-stemming (reducing words too much) or under-stemming (not reducing them enough). To address the limitations of stemming, lemmatization techniques are often used. Using vocabulary and morphological analysis, a lemmatizer tries to infer the dictionary form (lemma) of words based on their intended meaning. There are several common lemmatization techniques:

Lowercase conversion

This consists of converting all words to lowercase.

Synonym replacement

This technique involves replacing words with one of their synonyms.

Contractions removal

Contractions are words written as a combination of a shortened word with another word. Contraction removal consists of transforming the words in a contraction into their full-length form, e.g., “she’d invest in stocks” becomes “she would invest in stocks.”

Standardization (normalization) of date and time formats

For example, dates are converted to YYYYMMDD format, and timestamps to YYYYMMDDHH24MMSS.

NOTE

NER is highly sensitive to data preprocessing, where even minor changes can significantly impact the results. It's essential to carefully assess the consequences of each preprocessing step. For example, converting all words to uppercase could disrupt rules dictating entity characteristics, such as the expectation that country names begin with uppercase letters.

Entity extraction

During entity extraction, an algorithm is applied to a corpus of clean text to detect and locate candidate entities. In this step, the NER system designer should know which type of entities they are looking for in the text. The extraction process is a *segmentation* problem, where the goal is to find all meaningful segments of text that represent an entity. In this case, the name “Bank of England” needs to be identified as a single entity, even if the word “England” could also be a meaningful entity.

Since the goal of this step is to locate references to an entity, it might produce correct yet imperfect results. For example, unnecessary tokens might be included, as in “Banking giant JP Morgan Chase”. In other cases, some tokens might be omitted, such as missing “Inc.” in “JP Morgan Chase Inc.” or “Michael” in “Michael Bloomberg.”

Entity categorization

Once all candidate entities in the text have been extracted, the next step is to accurately map each valid entity to its corresponding entity type. For example, “Bank of America” should be classified as a company (COMP), “United States” as a country (LOC), “Bill Gates” as a person (PER), and any other token should be labeled as “O” to indicate that it is not a relevant entity.

The main challenge in this step is language ambiguity. For example, the words *bear* and *bull* are frequently used to indicate two species of animals. However, in financial markets, the word *bull* is often used to indicate an upward trend in the market, while *bear* describes a receding market.

Another example involves similar names that could refer to different entities. For instance, “JP Morgan” might describe the well-known financial institution JPMorgan Chase, but it could also refer to John Pierpont Morgan, the American financier who founded J.P. Morgan Bank.

To illustrate the NER process up to this step, we should be able to take a text such as...

*Gold prices rose more than 1% on Wednesday after the U.S. Federal Reserve flagged an end to its interest rate hike cycle and indicated possible rate cuts next year.*¹

...and produce a structured categorization, as illustrated in [Table 4-4](#). In this example, five types of entities were extracted: commodity (CMDTY), variable (VAR), nationality (NAL), organization (ORG), and miscellaneous (O).

Table 4-4. Outcome of entity extraction and categorization of a news title

entity_type	text
CMDTY	Gold
VAR	Prices
NAL	U.S.
ORG	Federal Reserve
O	rose more than 1% on Wednesday after the
O	flagged an end to its interest rate hike cycle and indicated possible rate cuts next year.

Entity disambiguation

If you aim to extend beyond merely extracting entities, which is crucial in numerous financial applications, you must proceed to disambiguate the identified and validated entities. This involves establishing a link between each correctly recognized entity in the data and its unique real-world counterpart.

The entity disambiguation step can present some challenges. One major issue is name variations. For example, a company can be mentioned in multiple ways, such as Bank of America, Bank of America Corporation, BoA, or BofA. Entity ambiguity is another challenge. For example, Bloomberg can refer to the company Bloomberg L.P. or its CEO, Michael Bloomberg. Finally, the knowledge bases used to disambiguate the entities might not always contain up-to-date information on all specific or novel entities that emerge in the market.

If we take our example, illustrated in [Table 4-4](#), adding entity disambiguation would result in real-world references, as illustrated in [Table 4-5](#). This example is illustrative, and more precise references could be used. For instance, the spot and future prices could be linked to a specific commodity exchange such as CME.

Table 4-5. Outcome of an entity extraction, categorization, and disambiguation of a news title

entity_type	text	reference
CMDTY	Gold	Chemical element with symbol AU
VAR	Prices	Spot price and future price on commodity exchanges
NAL	U.S.	Country in North America
ORG	Federal Reserve	Central Bank of the United States of America
O	rose more than 1% on Wednesday after the	
O	flagged an end to its interest rate hike cycle and indicated possible rate cuts next year.	

Evaluation

Evaluating the performance of NER systems in terms of their accuracy and efficiency is the last step in NER. An accurate NER system should detect and recognize all valid entities, correctly assign them to the appropriate entity types, and optionally link them to their real-world counterparts. Besides analytical performance, NER systems must also be assessed based on their computational efficiency, which includes runtime, memory consumption, storage requirements, CPU usage, and scalability to handle large-scale financial applications with millions of records.

To compute performance metrics for an NER system, four kinds of results are needed:

False positive (FP)

An instance incorrectly identified as an entity by the NER system

False negative (FN)

An instance that the NER system fails to classify as an entity, even though it is an actual entity in the ground truth

True positive (TP)

An instance correctly identified as an entity by the NER system

True negative (TN)

An instance correctly identified as a nonentity, consistent with the ground truth

These four values are often represented in a special tabular format known as a *confusion matrix*, as illustrated in [Figure 4-5](#).

		Actual values	
		Actually is an entity	Is not an actual entity
Predicted values	Recognized as entity	True positive (TP)	False positive (FP)
	Not recognized as entity	False negative (FN)	True negative (TN)

Figure 4-5. Confusion matrix of NER

NOTE

To compute the confusion matrix of a given NER model, you need to have a ground truth dataset with the actual values. The ground truth is mainly used for model training, where predicted values are compared against their true counterparts. This is usually a major challenge in NER, especially if you have big datasets. You, as a financial data engineer, will play a primary role in building and maintaining a labeled database to be used as the ground truth for NER systems.

Using the confusion matrix, the following performance evaluation metrics can be computed:

Accuracy

Accuracy measures the overall performance of the NER model and answers the question, “Out of all the classifications that were made, how many were correct?” In NER, this can be used as a measure of the ability of the model to distinguish between what is an entity from what is not. Accuracy works well as an evaluation metric if the cost of false positives and false negatives is more or less similar. This can be represented as a formula as follows:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

Precision measures the proportion of true positives to the number of all positives that the model predicted. It answers the question, “Of all instances that were classified as true positives, how many are correct?” In NER, this could be interpreted as the percentage of tokens (words or sentences) that were correctly recognized as enti-

ties out of all the tokens that are actually entities. A low precision value would indicate that the model is not good at avoiding false positives. Precision is a good measure when the cost of false positives is quite high. This can be represented as a formula as follows:

$$\textit{Precision} = \frac{TP}{FP+TP}$$

Recall

Recall measures the true positive rate of the model by answering the question, “Out of all instances that should be classified as true positives, how many were correctly classified as such?” Low recall indicates that the model is not good at avoiding false negatives. The recall is a good measure to use when the cost of a false negative is high. This can be represented as a formula as follows:

$$\textit{Recall} = \frac{TP}{TP+FN}$$

F1 score

The F1 score is a harmonic mean of precision and recall. It is widely used when the class representation in the data is imbalanced or when the cost of both false positives and false negatives is high. In financial NER, this is likely to be the case, as the vast majority of data tokens are not entities and the cost of mistakes is high. This can be represented as a formula as follows:

$$\textit{F1score} = \frac{2*(\textit{Recall}*\textit{Precision})}{\textit{Recall}+\textit{Precision}}$$

Additional evaluation metrics can be derived from the confusion matrix.² In many research papers on NER, the F1 score is used as the default metric. However, I highly recommend that you compute all four metrics to have an overview of your NER performance from different angles. For example, a low precision might tell you that you have a rule in your model that easily classifies a token as an entity. Similarly, a low recall might tell you that your model hardly classifies an entity as such; maybe your rules are too strict.

Now that you understand the necessary steps for developing an NER system, let’s explore the main modeling approaches that can be employed to build and operationalize an NER system.

Approaches to Named Entity Recognition

Numerous NER methods and techniques have been proposed in academic literature and by market participants. Frequently, these solutions are tailored or fine-tuned to suit particular domains. In this book, I will offer a taxonomy of seven modeling approaches: lexicon-based, rule-based, feature-engineering-based machine learning, deep learning, large language models, wikification, and knowledge graphs.

One thing to keep in mind is that these approaches aren't necessarily mutually exclusive. In many cases, especially when building complex NER systems, developers employ a combination of techniques. In the upcoming sections, I will discuss each of the seven approaches with some level of detail.

Lexicon/dictionary-based approach

This approach works by first constructing a lexicon or dictionary of vocabulary using external sources and then matching text tokens with entity names in the dictionary. A financial dataset, like reference or entity datasets, can function as a lexicon. Lexicons are flexible and can be tailored to any domain. For this reason, this approach could be a good choice for domain-specific tasks where the universe of entities is small or constant, or evolves slowly. Examples include sector names, financial instrument classes, and company names. Other examples might include accounting or legal texts, which rely on standard principles and formal language that doesn't change much over time.

Lexicons serve a dual purpose in NER. They can function as the primary extraction method or complement other techniques, as I'll illustrate later. Furthermore, a lexicon can be used for entity disambiguation. For example, a lexicon mapping company names to their identities can handle both recognition and disambiguation tasks.

The main advantages of lexicons are processing speed and simplicity. If you have a lexicon, then the extraction process can be viewed as a simple dictionary lookup. Keep in mind, however, that lexicons cannot recognize new entities that are not in the dictionary (e.g., new types of financial in-

struments). Additionally, lexicons are highly sensitive to the quality of data preprocessing and the presence of errors. As they cannot deal with exceptions or erratic data types, lexicons tend to guarantee better performance on high-quality data. Finally, lexicons might produce false positives if the context is not taken into account. For example, a stock ticker lexicon might contain the symbol AAPL for Apple, Inc. However, the abbreviation AAPL may also refer to “American Association of Professional Landmen” or “American Academy of Psychiatry and the Law.”

Rule-based approach

The rule-based approach employs a set of rules, created either manually or automatically, to recognize the presence of an entity in text. For example:

- *Rule N.1:* the number after currency symbols is a monetary value, e.g., \$200.
- *Rule N.2:* the word after Mrs. or Mr. is a person’s name.
- *Rule N.3:* the word before a company suffix is a company name, e.g., Inc., Ltd., Inc., Incorporated, Corporation, etc.
- *Rule N.4:* alphanumeric strings could be security identifiers if they match the length of the identifier and can be validated with a check-digit method.

Similar to the lexicon approach, rule-based methods tend to be domain-specific, making their transferability to other domains challenging. They are also particularly sensitive to data preprocessing issues, exceptions, and textual ambiguity, which can result in an large set of rules. Complex rule-based approaches are difficult to maintain, hard to understand, and can be slow to run. Therefore, they are recommended in cases where the language is either simple or subject to formal standards, such as accounting, annual reports, or SEC filings.

Feature-engineering machine learning approach

Lexicon- and rule-based methods commonly face challenges when complex data patterns need to be identified for accurate NER. In such cases,

modeling presents a compelling alternative. One prominent method involves feature-engineering machine learning, wherein a multiclass classification model is trained to predict and categorize words in a text. Being supervised, this approach requires the existence of labeled data for training.

To apply supervised machine learning, the modeler must select, and in most cases engineer, a set of features for each token.³ To give a few examples, features can be something like the following:

- Part-of-speech tagging (noun, verb, auxiliary, etc.)
- The word type (all-capitalized, all-digits, alphanumeric, etc.)
- Whether it's a courtesy title (Mr., Ms., Miss, etc.)
- The word match from a lexicon or gazetteer (e.g., San Francisco: City in California)
- Whether the previous word is a courtesy title
- Whether the word is a currency symbol (¥, \$, etc.)
- Whether the previous word is a currency symbol
- Whether the word is at the beginning or end of the paragraph
- Context aggregation features that capture the surrounding context of a word (e.g., the previous and subsequent n words)⁴
- Prediction of another ML classifier⁵

Once all relevant features have been carefully engineered, a variety of algorithms can be used. Among the most popular choices are logistic regression, Random Forests, Conditional Random Fields, Hidden Markov Models, support vector machines, and Maximum Entropy Models.

Feature-based models offer several advantages, such as speed of training and feature interpretability. However, several challenges might arise, such as the need for financial domain expertise, the complexity of feature engineering, difficulty modeling nonlinear patterns, and the inability to capture complex contexts for longer sentences. This is where more advanced machine learning techniques, such as deep learning, come into play, which I will introduce next.

Deep learning approach

In recent years, deep learning (DL) has established itself as the state-of-the-art approach for NER.⁶ DL is a prominent subfield of machine learning that works by learning a hierarchical representation of data via a neural network composed of multiple layers and a set of activation functions. A neural network can be thought of as a computational graph where each layer of nodes performs nonlinear function compositions of simpler functions produced at the previous layer. Interestingly, this process of repeated composition of functions has significant modeling power, which has contributed to the success of deep learning in solving complex problems.

There are several advantages to applying DL to NER. First, the modeler doesn't need to worry about the complexities involved in feature engineering, as deep neural networks are capable of learning and extracting features automatically. Second, DL can model a large number of complex and nonlinear patterns in the data. Third, neural networks can capture long-range correlations and context dependencies in the text. Fourth, DL offers high flexibility through network specifications (depth, width, layers, hyperparameters, etc.), which allows the modeling of a large number of domain-specific problems on large datasets.

A wide variety of network structures exist within the DL field. The ones that have shown remarkable success in NER-related tasks are Recurrent Neural Networks and their variants, such as Long Short-Term Memory, Bidirectional Long Short-Term Memory, and, most recently, attention mechanism-based models, such as Transformers.⁷

Deep learning is a powerful and advanced technique. However, I advise against using it by default for your NER task. DL models are hard to interpret and may require special hardware (e.g., a graphics processing unit, or GPU) and time to train. Try a simple approach first. If it doesn't work, then use more complex techniques.

Given the remarkable performance of complex models like DL in text-related tasks, development has extended to even more sophisticated models, such as large language models (LLMs), which I'll explore next.

Large language models

A large language model (LLM) is an advanced type of generative artificial intelligence model designed to learn and generate human-like text. Most LLMs leverage a deep learning architecture known as a Transformer, proposed in the seminal paper [“Attention Is All You Need”](#). Techniques such as Reinforcement Learning from Human Feedback (RLHF) are often used to align LLMs to human preferences. LLMs may also utilize other techniques such as transfer learning, active learning, ensemble learning, embeddings, and others.

LLMs are quite massive, often trained on vast amounts of text data and comprising millions or even billions of parameters. General-purpose LLMs are commonly known as *foundational models*, highlighting their versatility and wide-ranging applicability across numerous tasks. Prominent examples include OpenAI’s Generative Pre-trained Transformer (GPT) series, such as GPT-3 and GPT-4, Google’s BERT (Bidirectional Encoder Representations from Transformers), Meta’s Llama, Mistral, and Claude. LLMs are capable of performing a wide range of general-purpose natural language processing tasks, including text generation, summarization, entity recognition, translation, question answering, and more.

LLMs can also be fine-tuned to specific domains. Fine-tuning is the process of retraining a pre-trained LLM on a domain-specific dataset, allowing it to adapt its knowledge and language understanding to suit better the terminology, vocabulary, syntax, and context of the target domain. For example, the [FinBERT](#) is a domain-specific adaptation of the BERT model, fine-tuned specifically for the financial domain. It is trained on a vast amount of financial texts, such as news articles, earnings reports, and financial statements, to understand and process financial language and terminology effectively. FinBERT can be used for various tasks in the financial domain, including sentiment analysis, named entity recognition, text classification, and more.

LLMs can be a powerful technique for financial NER. This is because they are able to understand and process complex and domain-specific lan-

guage, recognizing entities such as financial instruments, accounting, and regulatory terms, as well as company and person names within the context of financial markets. For example, an LLM may be able to distinguish “Apple Inc.” as a tech company listed on NASDAQ from the word “apple” as a fruit, using contextual clues from surrounding text. They can also identify financial terms such as “S&P 100,” “NASDAQ Composite,” and “Dow Jones Industrial Average” as indexes rather than just random phrases. Similarly, LLMs may be able to distinguish between terms like “call option” and “put option,” understanding that they refer to specific types of financial derivatives, despite their similar structure.

Crucially, while LLMs may show outstanding performance in many financial language processing tasks, they can still encounter challenges with specialized and evolving financial terminology. For example, financial terms such as “interest rate swap” (CDS), “collateralized debt obligation” (CDO), and “mortgage-backed securities” (MBS) necessitate a deep understanding of financial instruments and their contexts. Similarly, terms such as “bonds” and “equity” have completely different meanings in finance than in the general sense. Furthermore, terms like “bitcoin,” “blockchain,” “cryptocurrency,” and “DeFi” (decentralized finance) have emerged relatively recently and require continuous model updates to stay current.

Retrieval-augmented generation (RAG) is an advanced technique employed to enhance the factual grounding, contextual relevance, and response accuracy of language models, especially in specialized domains like finance. RAG operates by retrieving relevant information from external sources, such as databases or documents, and incorporating this data into a language model's input prompt, thereby providing additional context to produce more accurate and contextually relevant responses.

In finance-specific tasks like financial NER, RAG enhances accuracy by disambiguating entities, staying up-to-date with rapidly changing information, and integrating domain-specific knowledge from financial documents and databases. This capability makes RAG particularly effective for identifying financial entities and handling complex jargon. Crucially, the success of RAG largely depends on the availability of reliable external data sources, highlighting its foundation in data engineering.

Another major challenge with LLMs is *hallucination*, which happens when an LLM generates irrelevant, factually wrong, or inconsistent content. Interpretability and transparency represent additional challenges, particularly in finance, where regulatory compliance and trust in decision-making are crucial.

Wikification

Wikification is an entity disambiguation technique that links recognized named entities to their corresponding real-world Wikipedia page.

[Figure 4-6](#) illustrates this technique through an example. In the first step (entity recognition), two entities (Seattle and Amazon) are identified. In the next step, the identified entities are linked to their unique matching Wikipedia page.

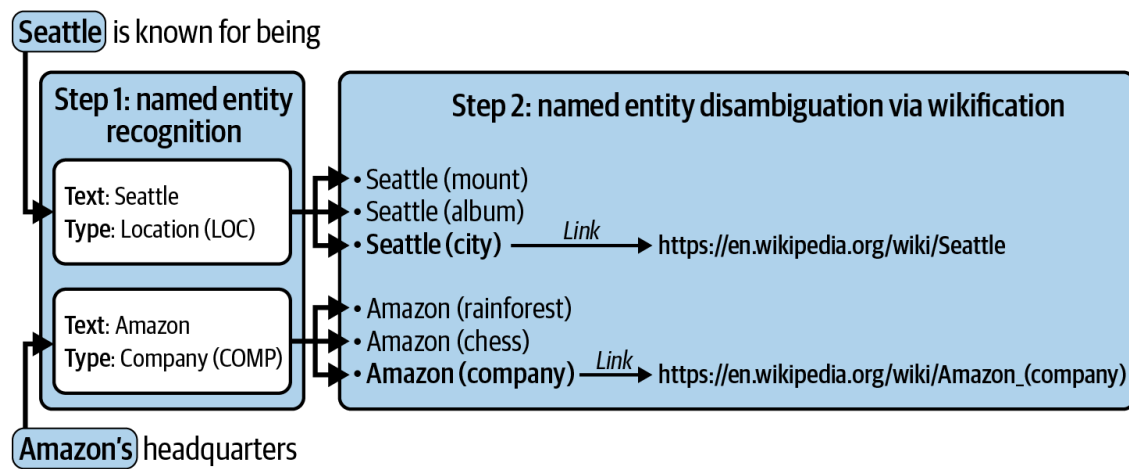


Figure 4-6. Wikification process

Several wikification techniques have been proposed, the majority of which utilize similarity metrics to determine which Wikipedia page is most similar to the recognized entity. One prominent implementation was first presented in [Silviu Cucerzan's groundbreaking work](#). Cucerzan proposed a knowledge base that incorporates the following elements:

Article entity/concept

Most Wikipedia articles have an entity/concept associated with them.

Entity class

Person, location, organization, and miscellaneous.

Entity surface forms

The terms used to reference the entity in text.

Contexts

Terms that co-occur or describe the entity.

Tags

Subjects the entity belongs to.

For example, the term [Berkeley](#) can refer to a large number of real-world entities, including places, people, schools, and hotels. Assume we are interested in identifying the University of California, Berkeley. In this case, the entity type is school or university; the context could be California, a

public university, or a research university; tags might include education, research, science, and others; and the entity surface form might be simply Berkeley.

An entity is disambiguated by first identifying its surface form.

Subsequently, two vector representations that encode contexts and tags are constructed: one for the Wikipedia context that occurs in the document and another for the Wikipedia entity. Finally, the assignment to a Wikipedia page is made via a process that maximizes the similarity between the document and entity vectors.

Knowledge graphs

Knowledge graphs have become an essential technique in internet-based information search and have been widely applied in entity disambiguation. There isn't yet a clear [definition of what a knowledge graph is](#). Still, it basically involves gathering different types of facts, knowledge, and content from many sources, organizing them into a network of nodes and links, and using it to provide more information to users upon submitting a search query. In other words, a knowledge graph can be thought of as a network of real-world entities—i.e., persons, locations, materials, events, and organizations—related together via labeled directed edges. [Figure 4-7](#) presents a simple illustrative example of a knowledge graph around the company Dell Technologies. The graph illustrates Dell Technologies and several related entities, such as its CEO, Michael Dell, and its supplier, Intel Corporation.

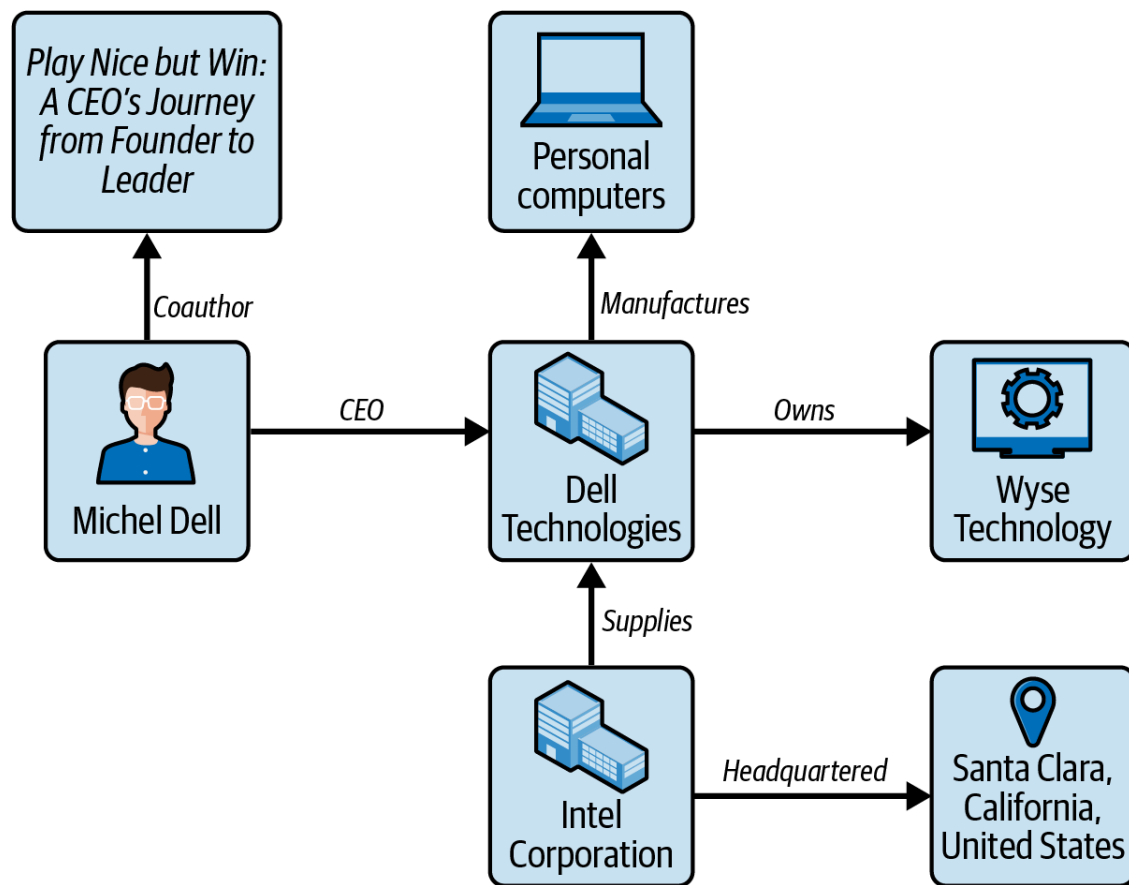


Figure 4-7. Illustrative example of a knowledge graph

The power of knowledge graphs stems from their extreme flexibility, which allows them to encompass a wide range of elements and interactions. This, in turn, can improve search results and reveal hidden data links that might otherwise go undetected using more traditional approaches.

Knowledge graphs have been proposed as an advanced approach to entity disambiguation within NER systems. A well-known implementation is the [Accurate Online Disambiguation of Named Entities](#), or AIDA. It constructs a “mention-entity” graph, where nodes represent mentions of entities found in the text, as well as the potential entities these mentions could refer to. These nodes are connected with weighted links based on the similarity between the context of the mention and the context of each entity. This helps the system figure out which entity the mention is most likely referring to. Additionally, AIDA connects the entities themselves with each other using weighted links. This allows AIDA to capture coherence among entities within the graph, aiding in the disambiguation process.

AIDA utilizes the *densest subgraph algorithm* to search the mention-entity graph. The densest subgraph algorithm helps identify the most densely connected subgraph within the larger graph. In the context of AIDA, this subgraph represents the set of mentions and entities that are most closely related to each other based on their connections and similarities. By identifying this densest subgraph, AIDA can determine the most coherent and relevant set of mentions and entities for a given context.

Two challenges may arise when finding such dense subgraphs. First, you need a reliable definition of the notion of a dense subgraph that ensures coherence and context similarity. Second, dense-subgraph problems are computationally expensive and almost NP-hard problems. This means that a heuristic or efficient algorithm is needed to guarantee a fast graph search to find the optimal dense subgraph.

Named Entity Recognition Software Libraries

Practitioners in industry and academia have created several software tools for NER. Several open source tools are available, including spaCy, NLTK, OpenNLP, CoreNLP, NeuroNER, polyglot, and GATE.

In addition to open source solutions, financial institutions and data providers build proprietary NER solutions. The most famous example is RavenPack analytics, which we discussed earlier in this chapter. Another prominent example is [NERD \(Named Entity Recognition and Disambiguation\)](#), developed by S&P Global's AI accelerator, Kensho. NERD is one of the few entity recognition and disambiguation tools tailored specifically for financial entities. NERD takes a text document as input and identifies mentions of named entities such as companies, organizations, and people. It also links the extracted entities to their real-world entries in the S&P Global comprehensive Capital IQ database.

FactSet provides a [Natural Language Processing API](#) that can be used to recognize and locate a wide range of entities in structured and semi-structured texts. This includes companies, people, locations, health conditions, drug names, numbers, monetary values, and dates. In addition to

NER, the API allows entity disambiguation by finding the best matching FactSet identifiers for companies and people found in the text.

Another tool that might be used for NER is *Automated Machine Learning* (AutoML). These solutions offer simple and user-friendly interfaces to automatically choose, train, and tune the best ML model/algorithm for a particular problem. One of the main advantages of AutoML is that it allows nonexperts to use sophisticated ML models. Examples of AutoML tools include open source libraries such as Auto-sklearn, AutoGluon, AutoKeras, and H2O AutoML, as well as cloud-based managed solutions such as Google AutoML and Amazon Sagemaker.⁸

AWS offers a specialized NLP AutoML service called Amazon Comprehend. Comprehend already has trained NER capabilities that you can immediately interact with, and it also offers the option to customize an NER system to your specific task (e.g., detecting financial entities). In addition, AWS introduced Bedrock, a managed service that allows users to build and fine-tune generative AI applications with foundation models.

Financial Entity Resolution

Once entities have been recognized and identified, a system should be available whereby the data associated with a unique entity in one dataset can be matched with data held in another dataset for the same unique entity. This process is very common in finance and is known as entity resolution (ER). In this section, you will learn what ER is and why it is important in finance. Then, you will learn how ER systems work and the different approaches to ER. Finally, I will present a list of software libraries and tools available for performing ER.

Entity Resolution Described

Entity resolution, also known as record linkage or data matching, refers to the process of identifying and matching records that refer to the same unique entity within a single data source or across multiple sources, particularly when a unique identifier is unavailable. When ER is applied to a

single dataset, it is often done to identify and remove duplicate records (*record deduplication*). When it is applied to multiple datasets, the goal is to match and aggregate all relevant information about an entity (*record linkage*).

Mathematically, let's represent two data sources as A and B and denote records in A as a and records in B as b. The set of records that represent identical entities in A and B can be written as:

$$M = \{(a, b); a = b; a \in A; b \in B\}$$

And the set of records that represent distinct entities as:

$$U = \{(a, b); a \neq b; a \in A; b \in B\}$$

As we will see later in this chapter, the main objective of an ER system is to distinguish the set of matches *M* from the set of non-matches *U*.

The Importance of Entity Resolution in Finance

Entity resolution is a common practice and represents a main challenge in the finance domain. As a financial data engineer, you will likely encounter the need to develop an ER system. Various industry initiatives have been established to address the financial ER problem. For instance, the [Financial Entity Identification and Information Integration \(FEIII\) Challenge](#) was initiated to create methodologies for aligning the various financial entity identification schemes and identifiers. Despite these efforts, the problem remains unresolved for several reasons, which I will outline next.

Multiple identifiers

As you learned in [Chapter 3](#), financial markets rely on a large number of data identification systems, each developed with a specific goal, structure, and scope. As such, it is typical that different financial datasets come with different identifiers. One financial identifier is typically sufficient to identify and distinguish unique entities when working with a single dataset. However, in many cases, people need to work with multiple datasets at once. For example, financial analysts or machine learning experts might

require a sample of data and features that span multiple data sources. To this end, different datasets might need to be merged via an ER system to create a comprehensive dataset for the analysis.

[Figure 4-8](#) illustrates a basic ER example where two datasets with different identifiers are matched. The table on the left contains six records identified by identifier B, while the table on the right holds data for the same records but uses identifier A. ER is performed by matching identifiers A and B, as depicted by the arrows. The resulting identifier mapping is as follows: 111 maps to BBB, 333 maps to AAA, and 222 maps to CCC.

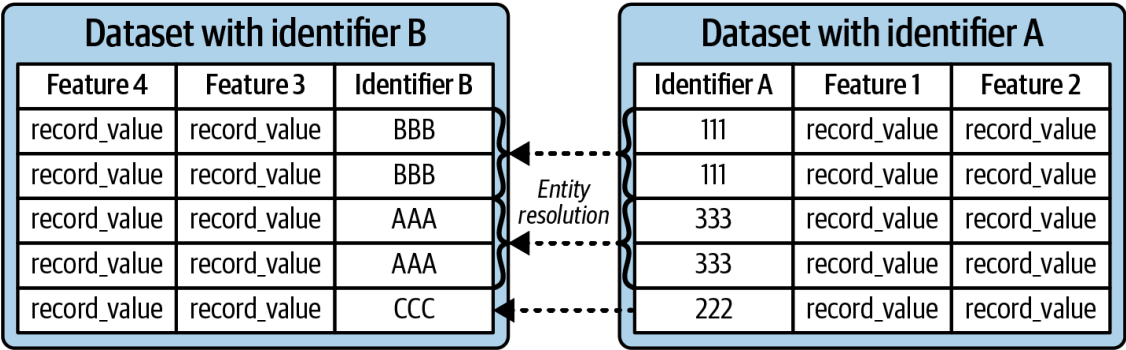


Figure 4-8. Entity resolution in the presence of two different identifiers

Keep in mind that if the datasets you want to merge use the same data identifier, then the task becomes a simple database join operation, and there would be no need to develop an ER system.

Missing identifiers

In some cases, a financial dataset may lack a proper identifier or may have an arbitrary identifier that does not match the specific one you need. For instance, data generated from nonregulated or decentralized markets, such as OTC, may not include appropriate data identifiers. A stock prices dataset might use the stock ticker as an identifier, while you may require the ISIN. Another common scenario involves agents engaged in financial activities who may intentionally obscure their identities to commit fraud. In such cases, an ER system is essential to identify entities based on the available data attributes. [Figure 4-9](#) illustrates the process of ER where identifiers are assigned to an unidentified dataset. The table on the right displays multiple features without entity identifiers. Using ER,

records are mapped to their corresponding identifiers, as indicated by the arrows.

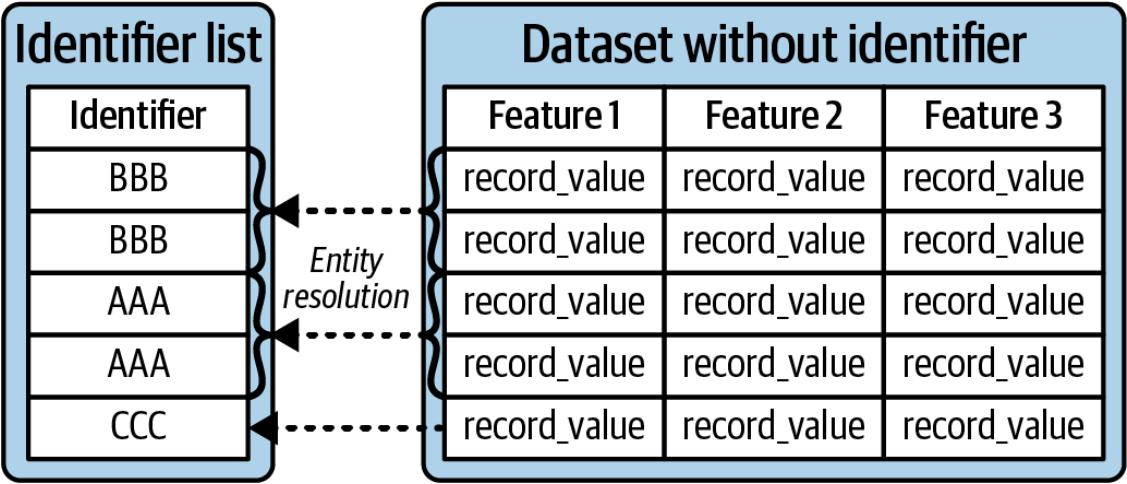


Figure 4-9. Entity resolution with unidentified data

One of the most important applications of ER in finance is fraud detection and identity verification. ER can help identify financial records that can be linked to the same real-life person or company using features such as name, email, bank account, country code, address, phone number, etc. Additionally, ER can identify anomalous activities when criminals attempt to conceal their identity and unlawful intentions by omitting crucial information or presenting it in an inaccurate manner.

One of the most common types of financial crimes is *money laundering*, an activity that makes illegally generated money look as if it comes from a legitimate source. A variety of money laundering schemes exist, and they continue to emerge over time. A typical example involves the same individual appearing as the owner of numerous companies, some of which provide no actual service but only shift money between different ends.

In banking, it is a common practice to verify the identity of an applicant when opening a bank account or conducting a financial transaction to ensure they are who they claim to be. This process is known as *know your customer* (KYC) and is aimed at preventing a pervasive form of fraud known as *identity fraud*. ER can be used for KYC to identify potential fraudsters who use different identities, email addresses, phone numbers, and other patterns to open new bank accounts and conduct financial transactions.

Data aggregation and integration

Information regarding various operations and activities within financial institutions is typically decentralized and scattered across multiple divisions. Data integration refers to the process of combining these multiple data sources to provide a comprehensive view of the organization. This process is highly relevant for financial institutions for purposes such as regulatory reporting and risk monitoring. In [Chapter 5](#), you will learn more about the importance of data aggregation in the financial sector.

To facilitate data integration, an ER system would be needed to match data across the different units and divisions within a financial institution. [Figure 4-10](#) provides a simple example illustrating this process. In this scenario, data originates from two divisions, 1 and 2. The data from each division is initially mapped to a common identifier before being merged into a single unified dataset.

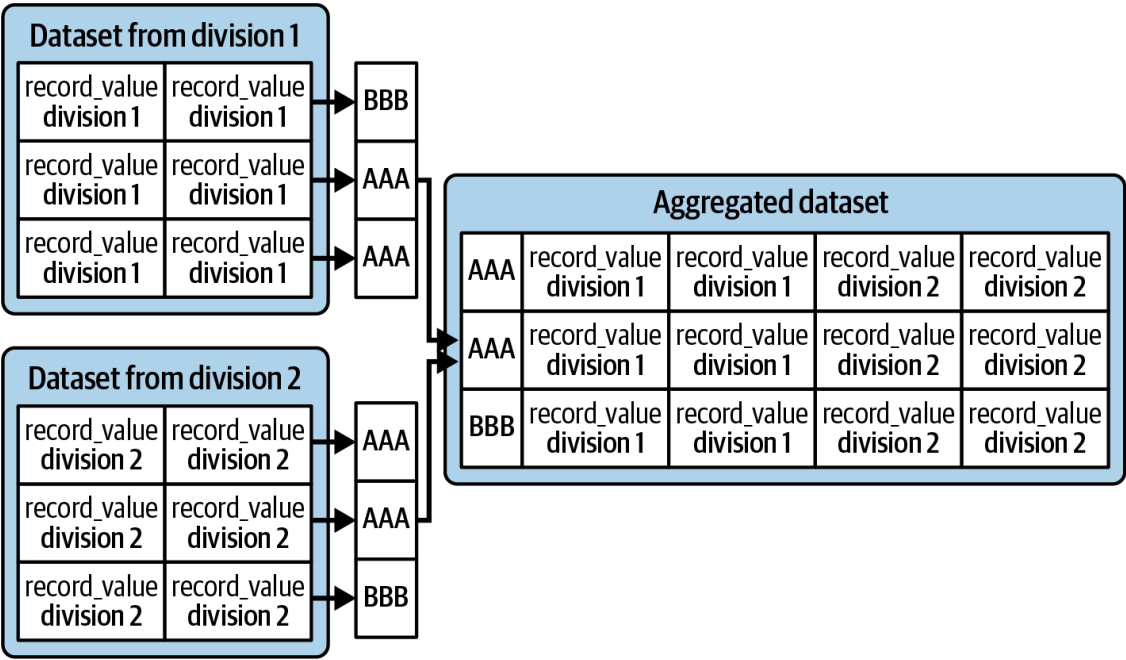


Figure 4-10. Entity resolution for data aggregation

Data deduplication

A frequent problem with financial data is the presence of duplicates, i.e., multiple records that convey the same information about an entity. Duplicate records are often encountered when using nonstandard identifiers such as person or company names, which can be recorded with multiple variations. [Chapter 5](#) will have a dedicated section detailing the problem of financial data duplicates.

The process of identifying and removing data duplicates is called data deduplication. Since deduplication requires matching similar entities in the same dataset, it can be treated as an ER problem. [Figure 4-11](#) shows an example illustrating this process. The table on the left contains two duplicate instances, (1,2) and (7,8). Using ER, it is possible to identify these duplicates and perform data deduplication, as shown in the table on the right.

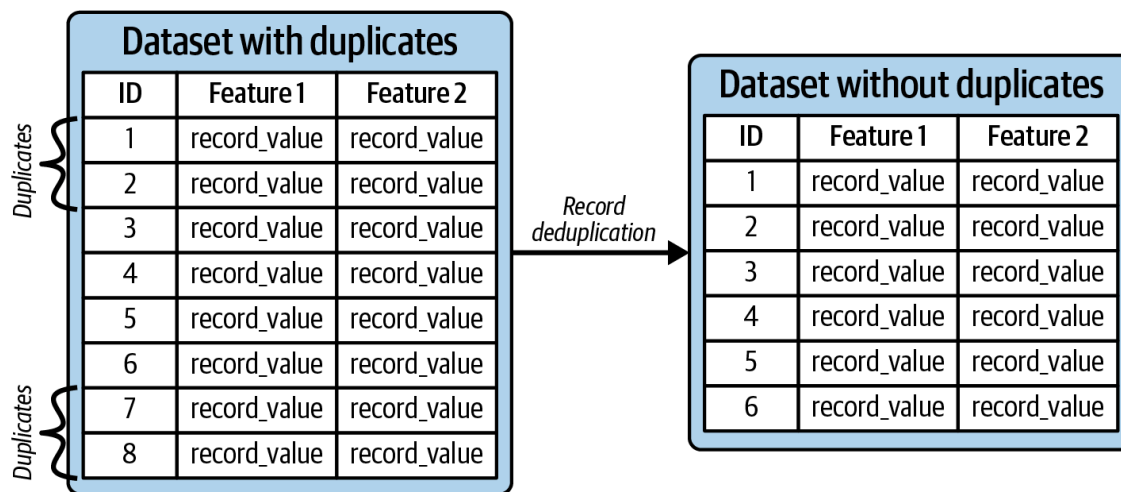


Figure 4-11. Entity resolution for data deduplication

How Does Entity Resolution Work?

A typical ER process involves five iterative steps, which I illustrate in [Figure 4-12](#). In the first step, *preprocessing* is applied to the input datasets to ensure their high quality for the task. The second step, *blocking*, is often required to reduce computational complexity when matching large datasets. In the third step, candidate pair records are *generated* and *compared* using a selected methodology. Successively, comparisons are *classified* into matches, non-matches, or possible matches. Finally, in the fifth step, the goodness of the matching process is *evaluated*. In the next few sections, we will explore each of these five steps in detail.

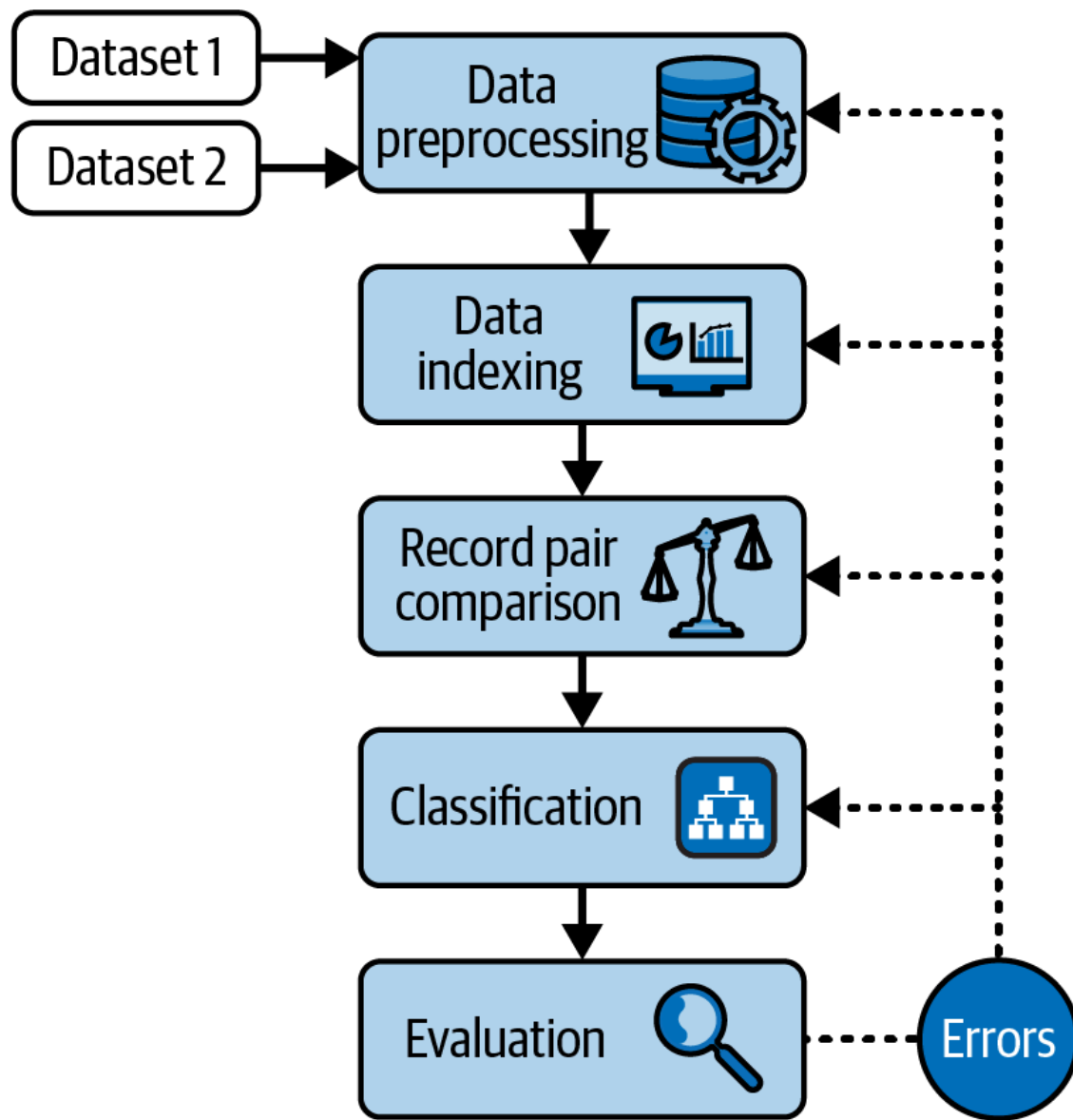


Figure 4-12. Entity resolution process

Data preprocessing

ER is highly sensitive to the quality of the input datasets. Therefore, before starting the matching process, it is crucial that the necessary rules are established and applied for quality assessment and data standardization. Such rules are particularly important for the data fields that will be used in the matching process, especially identifier fields. [Table 4-6](#) illustrates an example where three datasets store data about the same financial entity using different formatting styles.

Table 4-6. Nonstandardized data representations

	Entity name	Headquarter	Market capitalization	Ex- dividend date
Dataset 1	JP Morgan Chase	New York City	\$424.173B	Jul 05, 2023
Dataset 2	JPMorgan Chase & Co.	New York City, NY	\$424,173,000,000	2023-07-05
Dataset 3	J.P. Morgan Chase & Co.	New York	\$424,000.173M	5/7/23

As the table shows, the three records are the same but look different as they use different formats. Keep in mind that formatting heterogeneity may occur within the same dataset.⁹

To guarantee optimal data-matching results, data should be standardized using a consistent formatting method. The most common approach involves rule-based techniques, which employ a set of data transformation rules such as the following:

- Remove dots from entity names (e.g., J.P. Morgan Chase & Co. → JP Morgan Chase & Co).
- Remove stop words (e.g., The Bank of America → Bank of America).
- Expand abbreviations (e.g., Corp. → Corporation).
- Remove postfixes (e.g., FinTech firm → FinTech).
- Names should appear as “Given name, Surname”.
- Convert dates to the format “YYYY/MM/DD”.
- Parse fields into smaller segments (e.g., divide a field that contains full addresses like “270 Park Avenue, New York, NY” into multiple fields for the city, state, and street).
- Infer missing fields (e.g., zip code can be inferred from the street address).
- Remove duplicate records.

When performing data preprocessing, make sure you don't modify the original tables. Instead, make a new copy of the data and apply the transformations to it.

Indexing

Once the input datasets are cleaned and standardized, they should be ready for matching. In a typical scenario, the matching process will involve a comparison between each element in the first dataset with all elements in the second one. If the datasets at hand are small, then such a comparison can be done in a reasonable amount of time. However, with large datasets, the computational complexity may increase significantly. Consider a scenario where you want to match two datasets with 500k records each. If all pair-wise comparisons were to be performed, there would be a total of $500,000 \times 500,000$ or 250 billion candidate comparisons. Even at a processing speed of one million comparisons per second, it would still take 69 hours to match the two datasets. If both datasets have one million records each, then it will take around 11 days!

Crucially, in most ER problems, the majority of pair-wise comparisons will result in non-matches. This is because records in the first dataset often match a small subset of records in the second dataset. For this reason, it is common to observe that the number of pair-wise comparisons increases quadratically with the number of data records (i.e., $O(x^2)$, where x approximates the number of records in the datasets to match), while the number of true matches increases linearly.¹⁰

To overcome this issue, a number of data optimization techniques have been developed. Such techniques are often referred to as *indexing*, which aims to reduce the number of pair-wise comparisons needed by generating pair records that are likely to match and filter out the rest. The most common indexing technique is called *blocking*. It works by splitting the datasets to match into a smaller number of blocks and performing pair-wise comparisons among the records within each block only. To perform the splitting, a *blocking key* needs to be defined using one or more fea-

tures from the datasets. For example, a blocking key might place records in the same block if they have the same zip code or country.

Blocking presents a few challenges. First, it is highly sensitive to data quality. Small [variations in the data](#) might lead a blocking key to place a record in the wrong block. Second, blocking might entail a [tradeoff between computational complexity and block granularity](#). By defining a very specific blocking key, you will end up with many blocks, which is good for performance. But this comes at the risk of excluding true matches. On the other hand, using a more generic blocking key could result in a small number of blocks, which will lead to a large number of pair-wise comparisons that increase computational complexity.

[Figure 4-13](#) illustrates a simple blocking process. In this example, we have two datasets, A and B, that contain company information such as the market capitalization, the headquarters' country, and the exchange market on which the company is listed. If we were to perform all pair-wise comparisons, we would need to do $6 \times 6 = 36$ comparisons. However, using blocking criteria that group records in blocks based on the headquarters' country and exchange market, we reduce the number of pair comparisons to five.

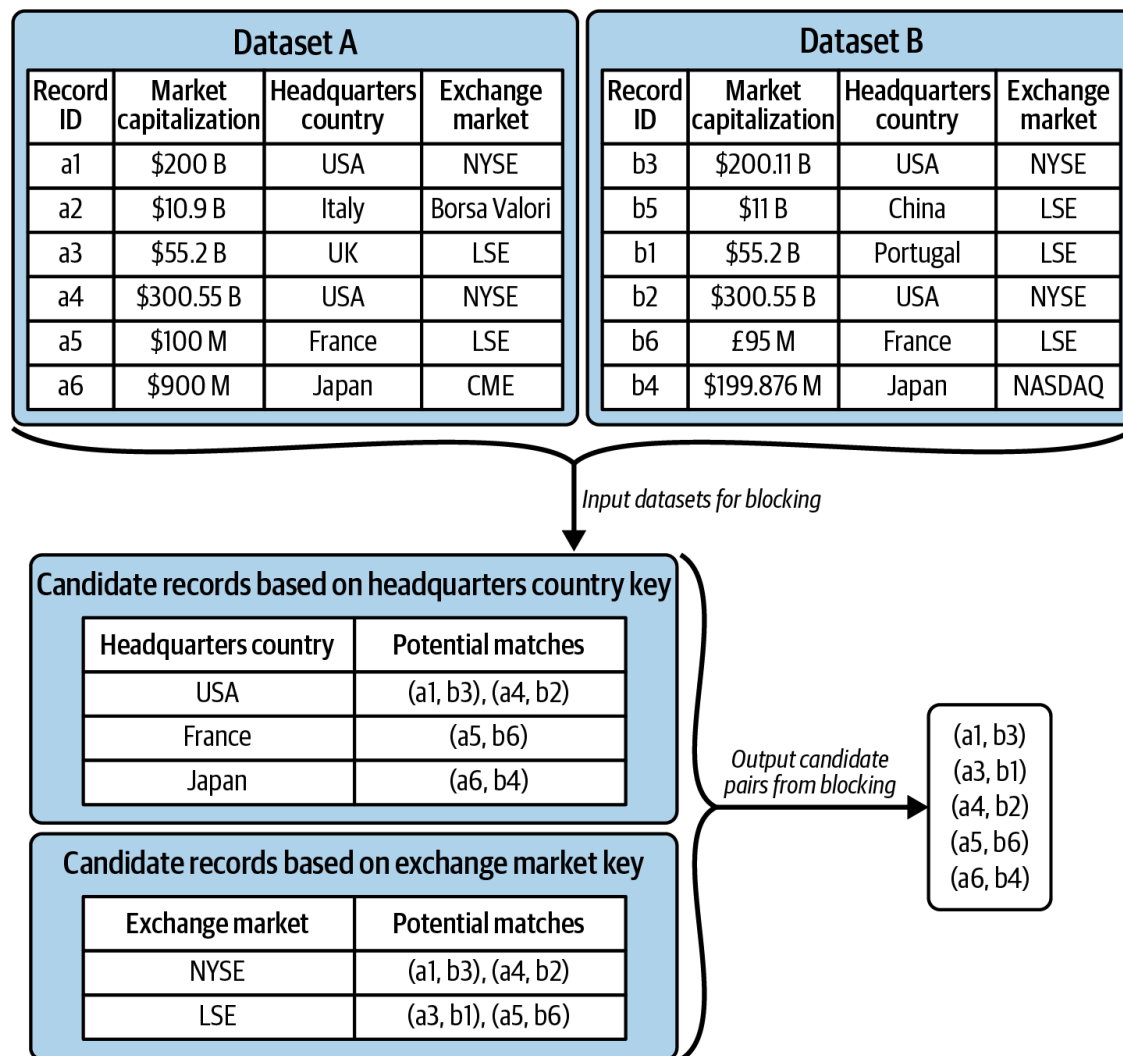


Figure 4-13. A simple blocking process

In addition to blocking, a number of other indexing techniques have been developed. Examples include Sorted Neighborhood Indexing, Q-Gram-Based Indexing, Suffix Array-Based Indexing, Canopy Clustering, and String-Map-Based Indexing.

Comparison

Once the candidate pairs have been generated, the next step involves the actual comparison between the records. The traditional approach to record comparison is based on pair similarity. This is often performed by aggregating all features into a single string and then comparing the string similarity between the pairs. Alternatively, comparing pair features individually by computing their similarities and combining them into a single similarity score is also possible.

Generally speaking, similarity scores are normalized to be between 0 and 1. A pair has a perfect match if its similarity score is 1, whereas a non-match is indicated by a score of 0. The comparison is called *exact matching* if it only allows for either a match or a non-match. Crucially, it is normal for similarity ratings to fall within the 0–1 range, in which case the matching is *approximate* or *fuzzy*. Approximate matching may occur due to differences in the datasets, such as the number of features (one dataset has a feature that the other does not), different formats (e.g., values reported in different currencies), information granularity (i.e., one dataset has a more granular identifier than the other), and information precision (one dataset rounds values to two decimals while the other uses three).

During the comparison phase, there are three types of matching scenarios:

One-to-one

Each record in the first dataset can only have one match in the second dataset (e.g., matching the same financial transaction in two datasets).

One-to-many

One record in the first dataset may have numerous matches in the second dataset (e.g., matching all transactions in one dataset associated with a specific credit card in another dataset).

Many-to-many

Numerous records from the first dataset can be matched to multiple records from the second dataset (e.g., matching multiple transactions within a trade recorded in a broker's database with transactions recorded by the clearing house or stock exchange).

As an illustrative example, [Table 4-7](#) shows the similarity scores for the five candidate pairs from [Figure 4-11](#). Records are first standardized (numbers expressed without decimals or multiples; all letters are uppercase), and then concatenated in a single string. Successively, the similarity is calculated between the concatenated strings using the *Longest Common Substring* (LCS) algorithm.¹¹

Table 4-7. Illustration of record comparison

Record pair	Pair string	Similarity score
(a1, b3)	a1: "\$200000000000USANYSE" b3: "\$200110000000USANYSE"	0.9
(a3, b1)	a4: "\$552000000000UKLSE" b1: "\$552000000000PORTUGALLSE"	0.75
(a4, b2)	a4: "\$300550000000USANYSE" b2: "\$300550000000USANYSE"	1
(a5, b6)	a5: "\$1000000000FRANCELSE" b6: "£95000000FRANCELSE"	0.81
(a6, b4)	a6: "\$900000000JAPANCME" b6: "\$199876000JAPANNASDAQ"	0.51

In addition to the LCS algorithm, there are several other methods available for computing pair similarities. These include Jaro–Winkler approximate string comparison, Levenshtein distance, edit distance, Jaccard similarity, Q-gram distance, and more.

Classification

Once all similarities have been computed, the next step is the classification of the candidate pairs into matching categories. In its most basic form, classification is binary: match or non-match. However, a less restrictive approach allows for three classes: match, non-match, and potential match. In either case, a match indicates a pair that refers to the same real-world entity in both datasets, while a non-match means that records in the pair refer to two different entities. A potential match is a pair of records that are likely to be a match but require a final clerical review for confirmation.

A variety of pair classification methods have been proposed, including the threshold-based approach, rule-based approach, probabilistic approach, and machine learning approach. Later in this chapter, we will discuss these models in more detail. To make a simple example, let's use a basic threshold-based approach to classify the results of the previous step

(comparison) that were reported in [Table 4-7](#). Let’s assume that a match has a similarity score greater than or equal to 0.9, a potential match has a score of 0.8 and above, and anything below 0.8 is a non-match. Using this approach, the outcome of the classification is illustrated in [Table 4-8](#).

Table 4-8. Illustration of a threshold-based pair classification

Record pair	Similarity score	Classification
(a1, b3)	0.9	MATCH
(a3, b1)	0.75	NON-MATCH
(a4, b2)	1	MATCH
(a5, b6)	0.81	POTENTIAL MATCH
(a6, b4)	0.51	NON-MATCH

Evaluation

The final step in an ER process is performance evaluation. A highly performant ER system is able to find and correctly classify all valid matches in the input datasets. Additionally, it needs to ensure computational efficiency in terms of runtime, memory consumption, storage needs, and CPU usage.

In most cases, ER systems are implemented for real-world financial applications; therefore, they need to scale to large applications with millions of records. Measuring computational complexity (e.g., in terms of $O()$ notation) is fundamentally important, even if optimization techniques such as indexing are applied. This is especially important when developing a streaming-based real-time record linkage system. In this case, complexity metrics and disk and memory usage figures can orient the implementation in terms of hardware, data infrastructure, and algorithmic optimizations. Additionally, as proposed by Elfeky et al. in their [research paper](#), performance can be measured in terms of the effectiveness of indexing techniques in reducing the number of record pairs to be matched (*reduction ratio*) while at the same time capturing all valid matches (*pair completeness*).

To evaluate the quality of the matching results of an ER system, a common practice is to use the binary classification quality metrics employed in machine learning and data mining, which we used for evaluating NER systems. In building such metrics, four numbers need to be calculated. *True positives* are the number of pairs correctly classified as matches, while *true negatives* are pairs correctly classified as non-matches. Similarly, *false positives* are non-matches that were mistakenly classified as matches, while *false negatives* are pairs that were classified as non-matches, but in reality, they refer to actual matches. [Figure 4-14](#) shows the confusion matrix representation of these figures.

		Actual values	
		Actually is a match	Actually is a non-match
Predicted values	Classified as match	True match True positive (TP)	False match False positive (FP)
	Classified as non-match	False non-match False negative (FN)	True non-match True negative (TN)

Figure 4-14. Confusion matrix of ER

Based on these four metrics, a variety of quality measures can be [calculated](#). For example, accuracy detects the ability of the system to make a correct classification (match vs. non-match). Precision measures the ability of the system to correctly classify true matches (i.e., how good the system is at avoiding false positives). Recall is another metric that measures the ability of the system to detect all true matches (i.e., how good the system is at avoiding false negatives). The F1 score is a harmonic mean of precision and recall and is used to find a balance between recall and precision.

Let's use our [Table 4-8](#) example to compute these four metrics. As illustrated in [Table 4-9](#), the final predictions are available in the column called "Predicted class after human review," while the ground truth values are available in the column "Ground truth class."

Table 4-9. Final ER classifications and their ground truth value

Record pair	Predicted class	Predicted class after human review	Ground truth class
(a1, b3)	MATCH	MATCH	MATCH
(a3, b1)	NON-MATCH	NON-MATCH	NON-MATCH
(a4, b2)	MATCH	MATCH	MATCH
(a5, b6)	POTENTIAL-MATCH	MATCH	MATCH
(a6, b4)	NON-MATCH	NON-MATCH	MATCH

From the data in [Table 4-7](#), we can compute the confusion matrix values as follows:

- TP: 3
- TN: 1
- FP: 0
- FN: 1

Then, we can compute the four quality metrics, as illustrated in [Table 4-10](#).

Table 4-10. Computed quality metrics

Quality measure	Value
Accuracy	0.8
Precision	1
Recall	0.75
F1 score	0.85

As a general performance metric, an accuracy of 0.8 is not bad, but it wouldn't be ideal in a critical application. The precision value of 1 tells us that the model doesn't produce false positives; if a pair is classified as a match, then it will be a match with 100% certainty. Recall tells us that the

model couldn't find all true matches and made a few false negative classifications. The F1 score of 0.85 shows an OK model performance, but one that is still not ideal for a good ER system.

Approaches to Entity Resolution

Numerous ER techniques have been proposed in the literature and by market participants. Such techniques are often named and classified differently; therefore, I summarize them into three categories: deterministic linkage, probabilistic linkage, and machine learning. These aren't necessarily mutually exclusive, and they can be combined to build an ER system. For example, a simple rule-based approach can be used to match high-quality records, while a probabilistic or machine learning approach is used for records with poor data quality. In the following sections, I will illustrate each approach in some detail.

Deterministic linkage

The simplest ER technique, known as deterministic linkage, performs data matching via a set of deterministic rules based on the available data fields. Various deterministic linkage methods have been proposed, including link tables, exact matching, and rule-based matching, which I'll cover next.

Link tables

A link table contains a mapping between two or more data identifiers. If two datasets use different identifiers mapped in a link table, then the datasets can be matched via an SQL join operation between them and the link table. [Figure 4-15](#) illustrates this approach.

For financial applications, link tables need to be built with a *point-in-time* feature to keep track of the possibility that identifiers might change, get reassigned, or become inactive. To this end, a good financial link table would include additional information such as the start and end date of the link, the link status, and any additional comments. For example, [Table 4-11](#) illustrates a link table that contains three links, where only one link (a1, b55) is active and has no end date, while link (a4, a20) ended in

31-12-2007 as the stock got delisted, and link (199, b44) ended in 20-01-1995 because the company merged with another one.

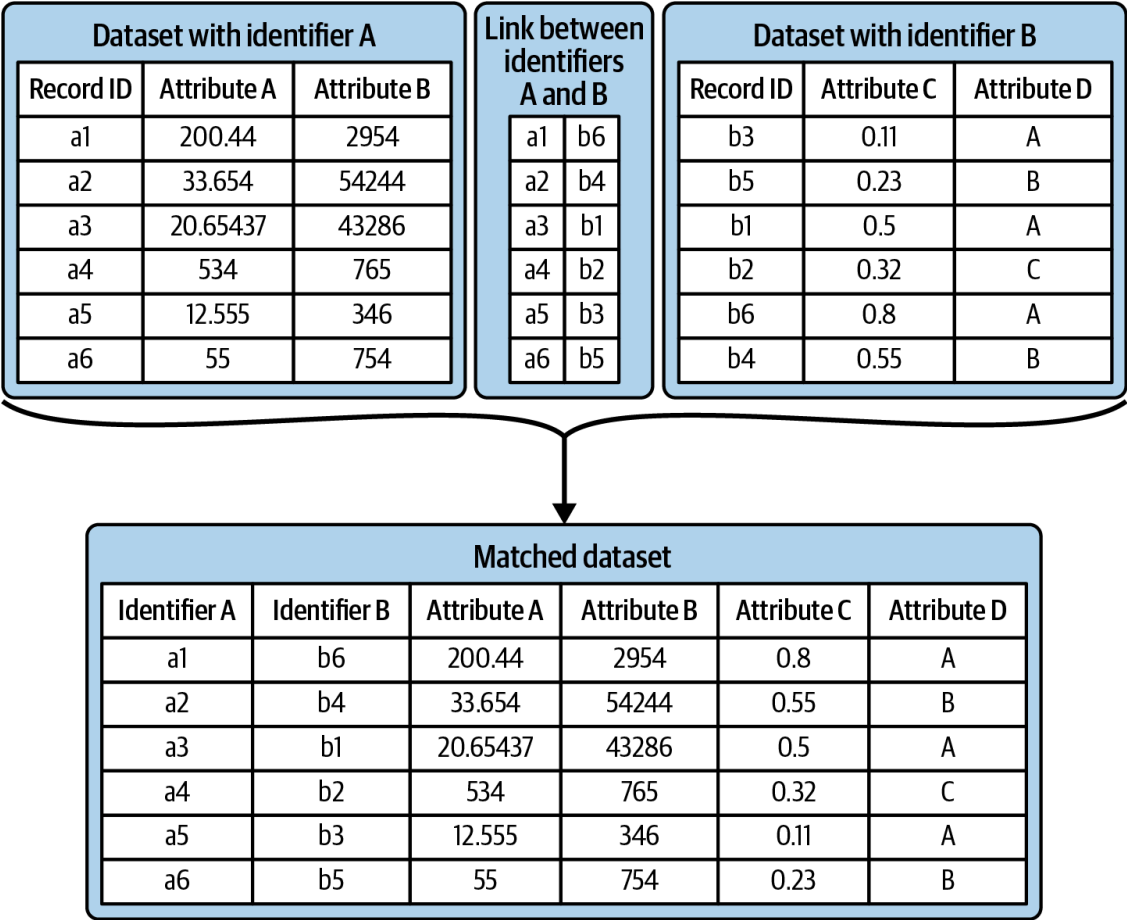


Figure 4-15. An ER process using a link table

Table 4-11. Example of a link table

Identifier A	Identifier B	Link start date	Link end date	Status	Comment
a1	b55	01-02-1990	-	Active	
a4	b20	20-01-2005	31-12-2007	Inactive	Stock delisted
a99	b44	20-01-1995	20-01-1995	Inactive	Merged with another company

The main advantages of link tables are simplicity, performance, and readability. However, they might be laborious to construct and require extensive maintenance and updating.

Financial institutions might create their own link tables internally. This is where you, as a financial data engineer, will play a major role.

Additionally, a variety of financial link tables are available as commercial products. This includes, for example, reference datasets that match different financial identifiers and other instrument characteristics. Another example is the famous data distributor Wharton Research Data Services (WRDS), which has created its own [linking suite](#) to enable users to link tables between the most popular databases on the WRDS platform.

Another notable example involves the series of initiatives [established](#) by the Global Legal Entity Identifier Foundation (GLEIF) in partnership with market participants to link the LEI with other financial identifiers. The result includes a list of open source link tables, such as the BIC-to-LEI, ISIN-to-LEI, and MIC-to-LEI mappings.

One of the most common use cases of entity resolution in finance is merging stock price data with company fundamentals data. If you have ever checked a financial news website, you will notice that data about stock close/open prices, bid/ask prices, and volume are available, together with fundamental data such as market capitalization and dividend distributions. This is done by matching data across price and fundamental datasets for the same entity.

A good example of a price/fundamentals ER system is the [CRSP/Compustat Merged Database \(CCM\)](#). The CCM database is a link table that matches historical events and market data from the CRSP database with company fundamentals data from S&P's Compustat database (both discussed in [Chapter 2](#)). As described by the vendor [documentation](#), the identifiers used in creating the link table are the following:

GVKEY

Compustat's company identifier.

ID

Compustat's issue identifier. One GVKEY may be associated with multiple GVKEYs.

PRIMISS

Compustat's primary security identifier.

PERMCO

CRSP's company identifier.

PERMNO

CRSP's issue identifier. One PERMCO may be associated with multiple PERMNOs.

The resulting link table matches all security identifiers with information on the link start date, link end date, CRSP identifiers, and Compustat identifiers.

Exact matching

In exact matching, records in two datasets are linked via a common unique identifier or via a [linkage key](#) that combines a set of data attributes into a single matching key. If a common unique identifier is available in both datasets, then the matching process becomes a simple SQL join operation on the unique key. The issue here is that financial datasets often use different identifiers. Additionally, an identifier may exist only from a certain point in time, and old records might lack identification. The same procedure can be followed with a linkage key, but instead of a unique identifier, a linkage key is constructed to merge the datasets. For linkage keys to provide good results, data must be of high quality (complete, standardized, deduplicated, and without errors).

Rule-based matching

A less restrictive approach to deterministic linking is the rule-based approach, where a set of rules is established to determine whether a pair of records constitutes a match. The primary benefits of this approach include the flexibility to define and incorporate rules, speed, interpretability, and simplicity. On the negative side, defining the rules may require considerable time and dataset-related domain knowledge. Moreover, as the datasets increase in complexity and vary in quality, you might end up with a large number of rules that can impact maintainability and performance.

A simple rule-based approach involves computing the similarity between records and classifying a pair as a match if it exceeds a given threshold (e.g., if the similarity is > 0.8 , then it's classified as a match; otherwise, it's a non-match). This method offers a good alternative to exact matching as it accommodates minor variations in the data attributes.

Probabilistic linkage

When a unique identifier is missing or the data contains errors and missing values, deterministic record linkage may deliver poor results.

Probabilistic linkage, also known as *fuzzy matching*, was developed to overcome this issue. Probabilistic methods have demonstrated superior linkage quality compared to deterministic approaches.

Probabilistic linkage takes a statistical approach to data matching by computing probability distributions and weights of the different attributes in the data. For example, assuming there are many fewer people with the surname “Bloomberg” than there are people with the surname “Smith” in any two datasets, the weight given for the agreement of values should be smaller when two records have the surname value “Smith” than when two records have the surname value “Bloomberg.” This is because it is considerably more likely that two randomly selected records will have the surname value “Smith” than it is that they will have the surname value “Bloomberg.”

To formalize these concepts, a variety of probabilistic linkage techniques have been developed.¹² However, to illustrate the main idea, let’s take as an example the well-known [framework of Fellegi-Sunter \(a theory of record linkage\)](#). Fellegi and Sunter proposed a decision-theoretic linkage theory that classifies a candidate comparison pair into one of three categories: link, non-link, and possible link. Pairs are analyzed independently. In their analysis, Fellegi and Sunter demonstrated that optimal matching can be achieved via a threshold-based strategy of likelihood ratios under the assumption that the attributes are independent of each other. To illustrate the main idea, let’s first define what the likelihood ratio is.

Let λ represent the agreement/disagreement pattern between two records in a given pair. Agreement can be expressed as a binary value (0 or 1) or, if needed, using more specific values. Using a binary agreement scale, if we have three attributes, then λ can be (1,1,1) if both records agree on all attributes, (1,1,0) if they agree on the first two but not the third, and so on. Let’s denote the set of all possible agreement patterns by

δ . For example, our three attributes can be represented in $\delta = \mathbf{8} (2 \times 2 \times 2)$ agreement patterns.

Let's assume we have two datasets we want to match, A and B. We create the product space as $A \times B$ to obtain all possible comparison pairs (assume we don't do indexing, for the sake of simplicity). Then, we partition the product space into two sets: matches (M) and non-matches (U).

Denote by $P(\lambda \in \delta \mid s \in M)$ the probability of observing the agreement pattern λ for a pair of records that are actually a match, and $P(\lambda \in \delta \mid s \in U)$ the probability of observing λ for a pair of records that is not a match. The likelihood ratio is then defined as:

$$R = \frac{P(\lambda \in \delta \mid s \in M)}{P(\lambda \in \delta \mid s \in U)}$$

For example, if we consider our three attributes to be market capitalization, exchange market, and name, then the likelihood of a pair in full agreement can be written as:

$$R = \frac{P(\text{agree on capitalization, agree on name, agree on exchange} \mid s \in M)}{P(\text{agree on capitalization, agree on name, agree on exchange} \mid s \in U)}.$$

If they agree on all attributes but the exchange, then the likelihood is:

$$R = \frac{P(\text{agree on capitalization, agree on name, disagree on exchange} \mid s \in M)}{P(\text{agree on capitalization, agree on name, disagree on exchange} \mid s \in U)}$$

The ratio R is referred to as *matching weight*. Based on likelihood ratios, Fellegi and Sunter proposed the following decision rule:

- If $R \geq t_{upper}$, then call the pair a link (match).
- If $R \leq t_{lower}$, then call the pair a non-link (non-match).
- If $t_{lower} < R < t_{upper}$, then call the pair a potential link.

For details on how to calculate the probabilities and thresholds, I refer the reader to the seminal work of Thomas N. Herzog, Fritz J. Scheuren, and William E. Winkler, *Data Quality and Record Linkage Techniques* (Springer).

Supervised machine learning approach

A limitation of deterministic and probabilistic approaches is that they tend to be specific to the datasets at hand and fail when there are complex relationships between the data attributes. Machine learning approaches excel in this area, as they are mainly focused on generalization and pattern recognition.

The supervised machine learning approach to record linkage trains a binary classification model to predict and classify matches in the datasets. As a supervised technique, it requires training data containing the true match status (match or non-match). Once trained on the labeled data, the model can be used to predict new matches for unlabelled data. Tree-based models,¹³ support vector machines,¹⁴ and deep learning¹⁵ techniques are among the most popular machine learning approaches used in ER.

Developing a supervised machine learning model for ER can be quite challenging. First, the model needs to consider the imbalanced nature of the data-matching problem, where most pairs correspond to true non-matches, while only a small fraction are true matches. Second, obtaining labeled training data can be quite challenging and time-consuming, especially for large datasets. Third, labeled data may not be available or accessible due to privacy issues. To solve this issue, a special type of ER, called privacy-preserving record linkage, has been proposed.¹⁶ Finally, an ML-based approach to ER might present interpretability and explainability challenges, especially when employing advanced techniques such as deep learning and boosted trees.¹⁷

Entity Resolution Software Libraries

Entity resolution is a well-known problem with a lengthy history of development and application. Many software programs for ER have been developed by individuals and organizations. As of the time of writing this book, there are open source tools like fastLink, Dedupe, Splink, JedAI, RecordLinkage, Zingg, Ditto, and DeepMatcher. Additionally, on the com-

mercial side, several vendors offer ER tools and solutions such as TigerGraph, Tamr, DataWalk, Senzing, Hightouch, and Quantexa.

Summary

In this chapter, you learned about two primary challenges commonly encountered by financial institutions: named entity recognition (NER) and entity resolution (ER). NER entails extracting and identifying financial entities from both structured and unstructured financial datasets.

Conversely, ER focuses on the critical task of matching data pertaining to the same entity across multiple financial datasets.

The landscape of challenges and solutions in financial NER and ER is dynamic, evolving alongside data, technologies, and changing market requirements. To excel at these tasks and gain a competitive edge, it's essential that you stay current with the latest updates, methodologies, technologies, and industry best practices around financial NER and ER.

Consider exploring machine learning techniques and natural language processing tools, and enrich your financial domain knowledge to enhance the accuracy and efficiency of your NER and ER systems.

Looking ahead, the next chapter will present and discuss the critical problem of financial data governance, exploring concepts and best practices for ensuring data quality, integrity, security, and privacy in the financial domain.

¹ [Ashitha Shivaprasad and Sherin Elizabeth Varghese, “Gold Climbs Over 1% After Fed Signals End of Rate Hikes”, Reuters \(December 2023\).](#)

² Have a look at the confusion matrix [Wikipedia page](#) for more details.

³ For a detailed discussion on how to design features for NER, see Lev Ratinov and Dan Roth's article, [“Design Challenges and Misconceptions in Named Entity Recognition”](#), in *Proceedings of the Thirteenth Conference on Computational Natural Language Learning (CoNLL-2009)*: 147–155, and Rahul Sharnagat's

[“Named Entity Recognition: A Literature Survey”](#), *Center For Indian Language Technology* (June 2014): 1–27.

- 4** To learn more about context aggregation, see the method proposed in Hai Leong Chieu and Hwee Tou Ng’s [“Named Entity Recognition with a Maximum Entropy Approach”](#), in *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*: 160–163.
- 5** To learn more about this advanced technique, see Radu Florian, Abe Ittycheriah, Hongyan Jing, and Tong Zhang’s [“Named Entity Recognition Through Classifier Combination”](#), in *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*: 168–171.
- 6** For a good survey of the use of deep learning in NER, see Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li’s [“A Survey on Deep Learning for Named Entity Recognition”](#), *IEEE Transactions on Knowledge and Data Engineering* 34, no. 1 (January 2020): 50–70.
- 7** A good read on the use of Transformers for NER is offered by Cedric Lothritz, Kevin Allix, Lisa Veiber, Jacques Klein, and Tegawendé François D. Assise Bissyande in [“Evaluating Pretrained Transformer-Based Models on the Task of Fine-Grained Named Entity Recognition”](#), in *Proceedings of the 28th International Conference on Computational Linguistics* (2020): 3750–3760.
- 8** One thing to keep in mind is that AutoML may be too generic to deal with the peculiarities of NER. For more on this issue, see Matteo Paganelli, Francesco Del Buono, Marco Pevarello, Francesco Guerra, and Maurizio Vincini’s [“Automated Machine Learning for Entity Matching Tasks”](#), in the *Proceedings of the 24th International Conference on Extending Database Technology (EDBT 2021)*, Nicosia, Cyprus, March 23–26, 2021: 325–330.
- 9** For a good read on this topic, please see Erhard Rahm and Hong Hai Do’s article, [“Data Cleaning: Problems and Current Approaches”](#), *IEEE Data Eng. Bull.* 23, no. 4 (December 2000): 3–13.
- 10** For more on this topic, refer to Mikhail Bilenko, Beena Kamath, and Raymond J. Mooney’s [“Adaptive Blocking: Learning to Scale Up Record Linkage”](#), in the *Sixth International Conference on Data Mining (ICDM’06)* (IEEE, 2006): 87–96.
- 11** The LCS implementation used to compute the similarities is the Python `SequenceMatcher` class in the [difflib package](#).

- 12** For an overview on this topic, have a look at Olivier Binette and Rebecca C. Steorts' ["\(Almost\) All of Entity Resolution"](#), *Science Advances* 8, no. 12 (March 2022): eabi8021.
- 13** A good example is Kunho Kim and C. Lee Giles' ["Financial Entity Record Linkage with Random Forests"](#), in *Proceedings of the Second International Workshop on Data Science for Macro-Modeling* (June 2016): 1–2.
- 14** A good example is Peter Christen's ["Automatic Record Linkage Using Seeded Nearest Neighbour and Support Vector Machine Classification"](#), in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (August 2008): 151–159.
- 15** A good read on deep learning for ER is Nihel Kooli, Robin Allesiaro, and Erwan Pigneul's ["Deep Learning Based Approach for Entity Resolution in Databases"](#), in *Asian Conference on Intelligent Information and Database Systems (ACIIDS 2018)*, Lecture Notes in Computer Science, vol. 10752 (Springer, 2018): 3–12.
- 16** For a good overview on this topic, I recommend Aris Gkoulalas-Divanis, Dinusha Vatsalan, Dimitrios Karapiperis, and Murat Kantarcioglu's ["Modern Privacy-Preserving Record Linkage Techniques: An Overview"](#), *IEEE Transactions on Information Forensics and Security* 16 (September 2021): 4966–4987.
- 17** Some effort has been made in this direction, for example Amr Ebaid, Saravanan Thirumuruganathan, Walid G. Aref, Ahmed Elmagarmid, and Mourad Ouzzani's ["Explainer: Entity Resolution Explanations"](#), in the *2019 IEEE 35th International Conference on Data Engineering (ICDE)* (IEEE, 2019): 2000–2003.